

Analisi dei Requisiti e Complessità Computazionale (v5.1)

Il sistema è stato progettato per simulare il comportamento di un Database Relazionale su file di testo a geometria fissa. L'obiettivo primario è l'elisione dei colli di bottiglia indotti dall'I/O sul disco, garantendo la massima reattività del finto server municipale

1. Autenticazione e Anagrafica Utenti

Struttura Dati e Canali Fisici

- Tabella Hash ad indirizzamento aperto con risoluzione delle collisioni tramite Linear Probing (Ispezione Lineare), persistita in modalità binaria pura sui file database/Master_Files/users_idx.txt (indice degli slot) e database/Master_Files/users.txt (dati anagrafici completi).
- Geometria Hardware: Ciascuno slot dell'indice hash occupa una riga fissa regolata dalla macro USER_IDX_LINE (6 byte totali). Il file anagrafico centrale users.txt occupa righe fisse regolate da USER_LINE_TOTAL ($10(\text{USER_ID}) + \text{MAX_USERNAME} + \text{MAX_PASSWORD} + 1(\text{Role}) + 1(\backslash\text{n})$ byte).

Complessità Computazionale

- Caso Medio (Login e Registrazione): $O(1)$ in tempo costante.
- Caso Peggior (Tabella in saturazione): $O(k)$, dove k rappresenta la capacità corrente della Tabella Hash.

Razionale d'Uso ed Espansione Dinamica Ammortizzata

- La Tabella Hash ad indirizzamento aperto garantisce un accesso immediato ed esclude l'uso di strutture a puntatori sul disco, che risulterebbero distruttive per l'I/O. Per impedire il degradamento asintotico indotto dalle collisioni in presenza di molti utenti, il server monitora costantemente il Load Factor (Fattore di Carico) ad ogni registrazione.

- Al superamento della soglia critica del 70% di saturazione, viene invocata automaticamente la funzione `expand_users_index()`. Invece di eseguire micro-scritture frammentate sul disco, il server applica una strategia di allocazione ammortizzata e prealloca in Append un intero blocco di 50 nuovi slot vuoti (marcati con -1) regolati dalla costante `BLOCK_SIZE_USERS`. Questa espansione Lazy pulisce l'ispezione lineare e blinda permanentemente l'autenticazione al tempo costante $O(1)$.
-

2. Sessione Locale e Meccanismo di Undo (Area Cittadino)

Struttura Dati e Isolamento delle Risorse in Sessione

- **ADT ReportList:** Una struttura concatenata semplice dinamica allocata in RAM per la memorizzazione isolata delle segnalazioni di proprietà dell'utente.
- **ADT ReportStack:** Uno Stack statico preallocato in RAM, limitato rigidamente a un massimo di 10 elementi di tipo Report, operante con semantica LIFO (Last In, First Out).

Complessità Computazionale delle Fasi Locali

- Compilazione dello storico ed iniezione in lista: $O(\log n)$ per ciascun record estratto dal disco.
- Inserimento nativo di una nuova segnalazione in lista: $O(1)$ in tempo costante in testa.
- Ricerca lineare e isolamento permessi per la modifica: $O(m)$ nel caso peggiore, dove m rappresenta la dimensione istantanea della lista di sessione (struttura ReportList dinamica, mutable ed espandibile indefinitamente in RAM, limitata nella prassi operativa dal volume di segnalazioni inviate dall'utente).
- Push / Pop nello Stack di Undo per il ripristino: $O(1)$ costante ad accesso diretto su array statico.
- Riversamento selettivo al Logout (Flush RAM-to-Bench): $O(m)$ lineare fino al termine della lista di sessione.

Pipeline di Sicurezza Logica e Protezione dai Cambi di Permessi

L'architettura confina l'intera operatività del cittadino all'interno della RAM volatile, attuando un isolamento totale dei dati (*Information Hiding*) e blindando il sistema contro accessi o modifiche non autorizzate (*Unauthorized Data Tampering*).

Fase A: Compilazione Protetta dello Storico (Login)

All'accesso del cittadino, il client esegue il Flusso Invertito Consistente coordinando due operazioni sequenziali di estrazione per garantire la massima freschezza del dato prima della manipolazione locale:

1. **Iniezione da file Master (load_master_reports_to_list):** Il server interroga l'indice utenti su disco tramite ricerca binaria in $O(\log n)$, ricava le disk row associate e compie un salto fseek a passi da REPORT_MASTER_LINE byte nei tre canali storici. In questo istante scatta una verifica geometrica rigida: il client controlla il flag della cella posizionato al byte offset REPORT_MASTER_LINE - 2. Se e solo se la cella è contrassegnata come Active ('A'), il record viene estratto e inserito nella ReportList della sessione corrente. Se il flag è 'N' (Null), significa che il report ha subito una modifica recente ed è stato evacuato dal Master, quindi viene scartato per evitare di caricare dati obsoleti. Vengono importati sia i casi OPEN, sia i casi IN_PROGRESS o CLOSED in sola lettura per garantire una vista storica coerente.
2. **Aggiornamento da Cache Server BENCH (load_bench_reports_to_list):** Subito dopo, la routine scansiona orizzontalmente in $O(n)$ lineare il file transitorio reports_bench.txt a passi da REPORT_BENCH_LINE byte fino al contatore corrente degli elementi attivi. Se rileva un report di proprietà del cittadino autenticato, il programma applica un filtro di sbarramento immediato sul suo stato logico interno: se lo stato è diverso da DESTROYED (Stato 3), il client aggancia direttamente la variante calda della BENCH alla ReportList di sessione. Non vi è alcun rischio di collisione o duplicazione con i dati del passo precedente, poiché i record presenti in cache possiedono una cella Master speculare già invalidata a 'N' e non caricata nella prima fase. Se il report in BENCH risulta invece DESTROYED (eliminato logicamente), viene interamente deallocato in RAM, scomparendo dalla vista dell'utente prima del flush pesante.

Fase B: Ricerca Isolata e Modifica Progetta in RAM (L'Undo Stack)

Quando il cittadino richiede la modifica di una segnalazione tramite l'opzione 3 del menu, **il programma non interroga mai direttamente i file del database sul disco**. Il client richiede il codice del report ed esegue una ricerca lineare in $O(m)$ esclusivamente all'interno della propria ReportList locale in RAM tramite la funzione list_find .

- Se l'ID inserito appartiene a una segnalazione di un altro utente, la ricerca in RAM fallisce immediatamente restituendo un puntatore nullo, respingendo l'attacco ed impedendo lo scavalco abusivo dei permessi di scrittura.
- Se il report viene individuato ma il suo stato è diverso da OPEN (es. IN_PROGRESS o CLOSED), l'operazione viene interrotta sul posto.

Se il report è valido ed OPEN, prima di applicare le modifiche del buffer, il client invoca `stack_push(revert_stack, report_to_modify)`. Questa istruzione effettua una clonazione profonda (*deep copy*) del nodo originario integro, mettendolo al sicuro nello stack LIFO. Se l'utente richiede l'azione di Undo, la `stack_pop` estrae il punto di ripristino in O(1) costante, distrugge la variante alterata e reinserisce il report immacolato nella lista RAM.

Fase C: Riversamento Selettivo e Sincronizzazione in Cache (Logout)

Al momento del Logout (opzione 5), per evitare corruzioni contabili ed infiltrazioni di record orfani nel backend, il client attiva un filtro di sbarramento rigido durante la scansione sequenziale della lista RAM tramite l'iteratore nativo:

[SCANSIONE LISTA IN RAM AL LOGOUT]

Iterazione del nodo: `flush_iterator`

(Se Stato == OPEN)

(Se Stato == IN_PROGRESS / CLOSED)

Invia al server via:

Dealloca il nodo locale in sicurezza

`register_report_from_citizen_ram` tramite `free_report()` ed esegue il bypass.

Il record Master stazionario non viene
toccato e non subisce alterazioni.

Il backend esegue i check

antiduplicazione (Scenario 2 e 3)

aggiornando la cache BENCH.

Il client invoca la funzione di backend `register_report_from_citizen_ram` **solo ed esclusivamente per i report che si trovano in stato OPEN**. I casi già presi in carico (IN_PROGRESS o CLOSED), che erano stati caricati in RAM al solo scopo di rendering visivo dello storico, vengono saltati dal ciclo e deallocati localmente in sicurezza.

Questo filtro impedisce che un report in lavorazione venga erroneamente inviato al server, il quale – non trovandolo nel Master Open – cadrebbe nello Scenario 3 (Nuovo Inserimento), alterando i contatori generali `REG_IDX_NM_REPORT` e `REG_IDX_STAT_OPEN` del registro statico `system_total_report.txt` e inquinando il database con duplicati fantasma.

3. Ricerca, Avanzamento Pratiche e Modalità di Lettura

Struttura Dati e Architettura degli Indici AVL Permanenti

- Inorder Arrays su Disco: Indici sequenziali piatti binarizzati estratti tramite visite simmetriche In-Order da Alberi Auto-Bilanciati AVL (PATH_AVL_REPORT_ID e PATH_AVL_USER_ID).
- Geometria Hardware:
- Indice Report ID: Righe fisse da 22 byte (AVL_REPORT_ID_LINE) prive di spazi intermedi: [REPORT_ID(10)][STATUS(1)][DISK_ROW(10)]\n.
- Indice User ID: Righe fisse da 21 byte (AVL_USER_ID_LINE) prive di spazi intermedi: [USER_ID(10)][REPORT_ID(10)]\n.

Complessità Computazionale delle Operazioni di Accesso

text

1. Ricerca univoca via Report ID: $O(\log n)$ su File d'Indice
2. Ricerca non univoca via User ID: $O(\log n) + m$ (Espansione bilaterale)
3. Modifica / Invalidazione su file Master: $O(1)$ diretto tramite salto fseek
4. Scansione orizzontale in Cache Server: $O(n)$ lineare limitata al counter bench

Razionale e Meccanismo di Avanzamento Pratiche (Area Dipendente)

- Quando il dipendente comunale invoca la funzione `employee_change_report_status` per modificare lo stato o avanzare una pratica di una segnalazione specifica, il server applica una pipeline di ricerca multilivello ottimizzata per azzerare l'I/O:
Fase A (Ricerca in Cache Operativa): Il server scansiona prima in $O(n)$ lineare il file transitorio `reports_bench.txt` a passi da `REPORT_BENCH_LINE` byte, arrestandosi al contatore corrente (`counter_bench`). Se rileva il report, esegue la modifica sul posto aggiornando i registri in $O(1)$ ed esce subito. Se lo stato viene mutato in `DESTROYED`, il report viene marcato come tale in Bench: alla successiva ispezione o rendering a video, il sistema lo intercetta ma applica un filtro esplicito e non lo visualizza, isolando i dati sporchi.
Fase B (Ricerca Logaritmica e Invalidazione Master): Se il report è assente in cache, il server interroga l'indice Report ID su disco invocando `findReportId(target_id)`. Questa funzione esegue una ricerca binaria logaritmica reale direttamente sul file di testo ordinato, effettuando salti posizionali sul punto medio (`mid`) a passi da 22 byte in tempo $O(\log n)$ puro. Individuata la riga fisica (`disk_row`), il server apre il file Master specifico a `REPORT_MASTER_LINE` byte in modalità binaria non distruttiva

("rb+"), compie un salto fseek posizionale immediato in $O(1)$ all'offset `REPORT_MASTER_LINE - 2` e sovrascrive il flag di cella attiva 'A' marchiandolo come 'N' (Null), trasformando lo slot in un buco vuoto.

Fase C (Transizione e Re-immissione in Bench): Subito dopo aver generato il buco e aver spinto la riga nello stack dei buchi LIFO, il server aggiorna i registri di stato in $O(1)$ via `update_system_counters`, muta lo stato del report (es. in `IN_PROGRESS`) e lo scrive in Append in fondo alla cache `reports_bench.txt`, azzerando la sua vecchia `disk_row` a -1. Il contatore della cache cresce di 1 unità e, se tocca la capacità limite (`LIMIT_BENCH`), innesca in automatico il Flush Pesante, risanando l'intero database.

Meccanismi e Modalità di Accesso in Lettura dei Dati

Il programma offre tre canali distinti e speculari di rendering visivo a video, ottimizzati per mostrare sempre la verità assoluta del database:

- 1. Il Flusso Invertito Consistente (Storico del Cittadino): Per ricompilare lo storico in tempo reale senza consumare RAM sul server, il client esegue una triangolazione invertita. Interroga l'indice utenti su disco a passi da 21 byte tramite ricerca binaria in $O(\log n)$. Trovato il blocco, esegue un'espansione bilaterale, alloca ed espande un array dinamico in RAM tramite `malloc` e `realloc` predittive (base 20 slot con raddoppio geometrico) e raccoglie tutti i Report ID associati all'utente.
Per ogni ID, triangola con l'indice dei report, estrae la `disk_row` ed effettua il salto $O(1)$ nel Master: se la cella è contrassegnata come Active ('A'), inietta il record nella lista. Subito dopo, scansiona la cache BENCH in $O(n)$: se trova varianti fresche dello stesso utente, invoca `list_remove` sterminando la versione Master obsoleta e inserisce la variante della BENCH (se non è DESTROYED). Il rendering finale mostra lo storico pulito.
- 2. Visualizzazione Sequenziale per Stato (Dipendente): Sfrutta il controllo hardware del valore di ritorno di `fread` a passi da `REPORT_MASTER_LINE` byte. Legge i record dal Master selezionato e, se incontra un byte 'N', applica la clausola `continue` saltando lo slot in $O(1)$. L'arresto del ciclo `while` avviene in modalità autonoma non appena la `fread` rileva il raggiungimento del fine file hardware (EOF), eliminando righe sentinella orfane. Visualizza poi i record in BENCH corrispondenti allo stesso criterio.
- 3. Timeline Cronologica Ordinata (Coda a Priorità): Il dipendente comunale può ispezionare la linea temporale dei casi urgenti. Il sistema scansiona i Master operativi, scarta i buchi 'N' in $O(1)$ e inserisce i puntatori opachi nell'ADT `PriorityQueue` in RAM tramite la funzione `pq_enqueue`. L'immissione esegue una scansione lineare ordinata: prioritizza l'urgenza scalare decrescente ('2' -> '1' -> '0') e, a perfetta parità di urgenza, applica l'helper `compare_dates_fifo`

ordinando per stringa temporale remota. L'estrazione (pq_dequeue) restituisce la massima criticità comunale in tempo costante $O(1)$.

4. Gestione Spazio e Riciclo Chirurgico dei buchi in Master

Struttura Dati e Canali Fisici

- File ausiliari operanti con semantica di Stack LIFO: database/Master_Files/open_holes.txt, in_progress_holes.txt e closed_holes.txt.
- Geometria Hardware: Righe fisse da 11 byte (SYSTEM_REG_LINE), formattate in modalità binaria rigida come `%010d\n` per ospitare gli indici numerici delle righe liberate N o vuote.

Complessità Computazionale

- Inserimento indice buco (Push): $O(1)$ costante in Append.
- Estrazione e troncamento buco (Pop LIFO): $O(1)$ costante tramite calcolo matematico hardware.

Razionale d'Uso ed Eliminazione del Degrado da Frammentazione

- Nello sfoltimento di un archivio storico su file, la ricerca di spazi vuoti generati dalle rimozioni causa solitamente un degradamento lineare in $O(n)$. Per azzerare questo costo, il server non scansiona mai orizzontalmente i file Master alla ricerca di celle inutilizzate. Quando una riga viene marcata a 'N', il suo indice numerico viene spinto (Push) in coda al rispettivo file dei buchi.
- Al momento del Flush Pesante della cache BENCH, prima di inserire un report, il server invoca la funzione `pop_hole_index()`. Questa interroga lo stack dei buchi: si posiziona geometricamente sull'ultima riga scritta tramite `fseek`, estrae il valore della riga master libera e, per rimuovere l'indice consumato senza riscrivere il file, calcola la nuova dimensione decurtata di 11 byte ed esegue un troncamento hardware sul posto del descrittore binario (tramite `primitive_chsize` o `truncate`). Se lo stack ha restituito una riga valida, il server fa un salto `fseek` chirurgico sul Master open, pialla il vecchio carattere 'N' e sovrascrive lo slot vuoto in $O(1)$ costante, marchiando la nuova riga come Active 'A' ed impedendo l'espansione indefinita dell'archivio sul disco.
- Se `pop_hole_index()` non restituisce un indirizzo valido il report viene caricato in append alla coda del file di stato

5. Dashboard Statistica Comunale

Struttura Dati e Aggiornamento Atomico

- Registro di controllo binarizzato statico stazionato nel file di testo centrale database/Master_Files/system_total_report.txt.
- Geometria Hardware: Righe fisse regolate da SYSTEM_REG_LINE (11 byte per riga, formato %010u\n). Mappa un vettore statico rigido di 13 variabili pre-calcolate (ID unico, contatore BENCH, volume attivo totale, 3 contatori di stato, 5 ripartizioni di categoria e 2 contatori di consistenza dei nodi degli indici AVL sul disco).

Complessità Computazionale

- Lettura, Scrittura e Rendering della Dashboard: $O(1)$ costante ed istantaneo ad accesso posizionale.

Razionale d'Uso ed Elisione dell'I/O

- Per consentire alla municipalità di ottenere un quadro analitico istantaneo senza dover scansionare gigabyte di archivi sul disco ad ogni richiesta, il server aggiorna le metriche in tempo reale e in modalità lazy. Qualsiasi nuovo inserimento assoluto incrementa subito il contatore globale, lo stato OPEN e la categoria all'interno dello Scenario 3 di register_report_from_citizen_ram. Qualsiasi avanzamento effettuato dal dipendente invoca la funzione helper update_system_counters, che storna i vecchi stati ed incrementa i nuovi eseguendo salti fseek posizionali atomici moltiplicando l'indice del registro per 11 byte.
- Le funzioni operano tramite le primitive binarie pure fread e fwrite, azzerando i rischi di iniezione di caratteri di carriage return (\r\n) invisibili tipici delle funzioni orientate al testo su sistemi Windows, garantendo che ciascuna riga rimanga isolata e che la dashboard venga renderizzata a video in tempo costante ed immediato.